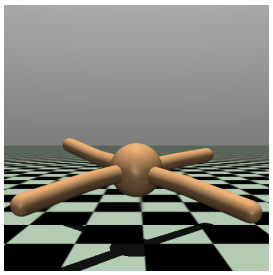
Coursework 2: On-Policy vs Off-Policy RL in Continuous Environments

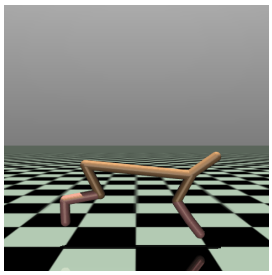
Exploring state-of-the-art continuous control Deep-RL algorithms

Due Date: **April 3, 2026 12:00 P.M.**

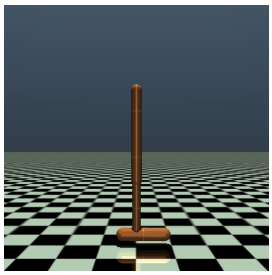
In this coursework, you will explore Proximal Policy Optimization (PPO) (Schulman et al., 2017) and Soft Actor-Critic (SAC) (Haarnoja et al., 2018), two modern state-of-the-art deep reinforcement learning algorithms widely used in robotics and beyond. You will apply these algorithms to **two** MuJoCo Gymnasium (Towers et al., 2024) environments of your choosing. MuJoCo Gymnasium provides 3D continuous control environments based on the MuJoCo physics simulator (Todorov et al., 2012). While these environments are toy examples, they are conceptually very close to modern robotics simulation environments.



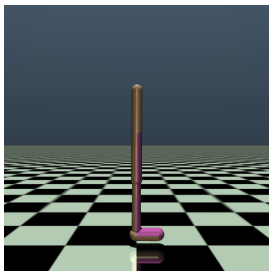
(a) Ant



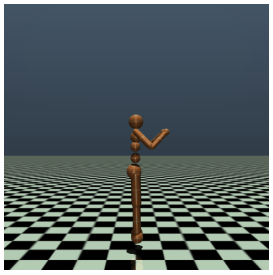
(b) Half Cheetah



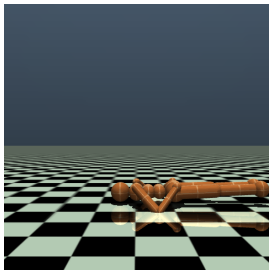
(c) Hopper



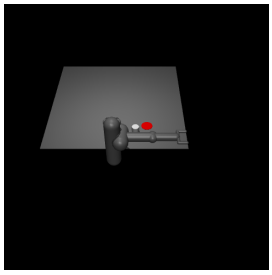
(d) Walker2d



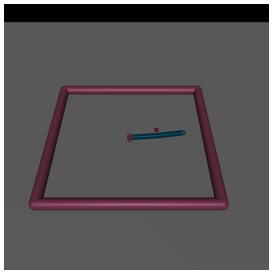
(e) Humanoid



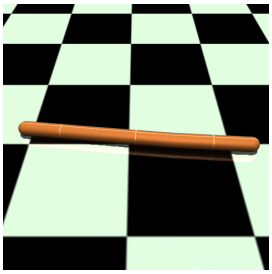
(f) Humanoid Standup



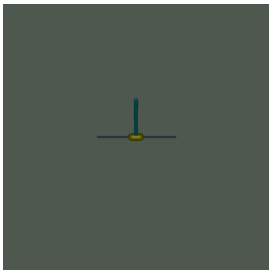
(g) Pusher



(h) Reacher



(i) Swimmer



(j) Inverted Double Pendulum

Figure 1: MuJoCo Gym environments used in experiments (excluding Inverted Pendulum).

This coursework mimics a research pipeline. Imagine you are tasked with choosing the best method for some particular task. To do this, you must benchmark the methods against each other. In the coursework you will do exactly this, except that you will also need to make sure you understand the methods thoroughly, as you must implement core logic yourself. To complete the coursework, you must:

- **Implement update steps**: Complete the parts of the update logic for PPO and SAC (Problem 1).
- **Choose environments**: Select 2 environments from Gymnasium's MuJoCo environments shown in Figure 1. You will run PPO and SAC on both of these environments and include the results in your report (Problem 2).
- **Test your implementations**: Verify that your implementations are correct by running them and checking that returns increase over time (Problem 2).
- **Tune hyperparameters**: Run a grid search over hyperparameters of your choosing to identify the best configuration (Problem 3).
- **Compare methods**: Use the provided plotting and rendering utilities to compare PPO and SAC, and analyze the performance of your final learned policies (Problem 4).
- **Write a report**: Detail your findings in a report formatted like a research paper section. This report, together with your code, will form the basis of your mark (Problem 5).

Code: You will find skeleton code in the following repository: https://github.com/LeoHink/rrl_mujoco_cw2. The repository contains a complete training pipeline; you are only responsible for implementing specific core logic (detailed in Problem 1). You do not need to understand or modify the rest of the codebase unless you are interested.

Collaboration: You may work in pairs or individually for this coursework.

Below we describe each of these steps as problems, which should guide you through the coursework in a logical order. We provide additional guidance on how to structure your report in Problem 5.

If at any point you run into issues with the code, you can ask for help through the following channels (in order of preference):

- Leave an issue on the GitHub (the public repository)
- Ask or raise issues during or after the tutorial
- Ask on Piazza

Getting Started: Cloning the Repository

You will find installation instructions in the `README.md` file of the repository. As a first step, clone the repository to a location of your choosing. You can also opt to mirror the repository as in CW1.

```
git clone git@github.com:LeoHink/rrl_mujoco_cw2.git
```

Make all the necessary changes and run the files locally on your computer. You will later be asked to upload your code to LEARN as part of your submission. If you want to use git and GitHub for version control, make sure to point git toward your own **private** repository so that you push changes there. One option is to mirror the repository as you did for CW1. If you are unsure how to do this, it is also fine to keep all changes locally, or ask for help.

Importantly, do not push your changes or make pull requests to the public repository!

1 Implement the Update Step

Your first task is to implement the update logic for both algorithms. In the `ppo.py` and `sac.py` files, you will see a handful of functions right above the main training loop, labeled with `# You Must implement these functions`. The functions you must implement are listed below.

Table 1: Functions to implement in `agents/ppo.py`

Function	What it computes
`compute_gae`	GAE advantages and return targets (backward TD recursion)
`compute_ratio`	Log-ratio and ratio $\pi_\theta/\pi_{\text{old}}$
`compute_policy_loss`	PPO clipped surrogate objective
`compute_value_loss`	MSE value-function loss

Table 2: Functions to implement in `agents/sac.py`

Function	What it computes
`compute_q_target`	Bellman TD target: $r + \gamma(1-d)\,(\min Q_{\text{target}} - \alpha \log \pi)$
`compute_actor_loss`	Policy loss: $\mathbb{E}[\alpha \log \pi - \min Q]$
`compute_alpha_loss`	Entropy coefficient tuning loss
`soft_update`	Polyak average: $\theta_{\text{target}} \leftarrow \tau\theta + (1-\tau)\theta_{\text{target}}$

You should consult the original papers or OpenAI's Spinning Up documentation for `PPO` and `SAC`. While many implementations exist online, we strongly recommend attempting the implementation from the theory first to solidify your understanding—think of it as guided, self-directed learning.

We have also included some theoretical context below which you may wish to reference as you work through the implementations.

Proximal Policy Optimization (PPO) – On-Policy

PPO is designed to take the largest possible step to improve the policy without "collapsing" performance. It achieves this using a **Clipped Surrogate Objective**.

The Objective Function

The core of PPO is the ratio between the new policy π_θ and the old policy $\pi_{\theta_{\text{old}}}$:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

Instead of a standard policy gradient, PPO uses a loss that prevents $r_t(\theta)$ from moving too far from 1:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon)\, \hat{A}_t \right) \right]$$

Variables:

- $\hat{A}_t$: The advantage estimate (computed via GAE), representing how much better action a_t was compared to the average action under the current value function.
- ϵ: Clipping coefficient (typically 0.1 or 0.2) that limits the policy update size. Not to be confused with ϵ in ϵ-greedy policies.

Generalized Advantage Estimation (GAE)

In your implementation, you will use GAE to compute the advantage $\hat{A}_t$. This involves the TD error δ_t at each timestep:

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

The GAE estimate can be computed via a backward recursion:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\, \hat{A}_{t+1}$$

starting from $\hat{A}_T = 0$ at the end of the trajectory. Unrolling this recursion gives the exponentially decayed sum of TD errors:

$$\hat{A}_t = \delta_t + (\gamma\lambda)\, \delta_{t+1} + (\gamma\lambda)^2\, \delta_{t+2} + \cdots$$

Variables:

- γ: The discount factor (e.g., 0.99), determining the agent's horizon.
- λ: The GAE parameter (e.g., 0.95), which trades off bias (low λ) vs. variance (high λ).
- $V(s)$: The value function estimate provided by the critic network.

PPO Update Pseudocode

1. Collect a set of trajectories $\mathcal{D}$ using the current policy $\pi_{\theta_{\text{old}}}$.
2. Compute advantage estimates $\hat{A}_t$ and return targets using GAE.
3. **For** K epochs **do**:
   - Sample a minibatch from $\mathcal{D}$.
   - Calculate $r_t(\theta)$ and the clipped loss L^{CLIP}.
   - Update policy parameters θ to maximize L^{CLIP}.[1]
   - Update value network ϕ to minimize MSE: $L^{\text{VF}}(\phi) = (V_\phi(s_t) - R_t)^2$.

The policy ratio $r_t(\theta)$ measures how much the current policy π_θ has shifted relative to the "old" policy $\pi_{\theta_{\text{old}}}$ that collected the data.

Soft Actor-Critic (SAC) – Off-Policy

SAC is an off-policy algorithm that maximizes not just the expected reward, but also the **entropy** of the policy. This encourages exploration and prevents the policy from prematurely collapsing to a single deterministic action.

The Value and Policy Objectives

SAC learns a Critic (Q_ψ) and an Actor (π_θ). The "Soft" Bellman target for the Critic includes an entropy term:

$$J_Q(\psi) = \mathbb{E}_{(s,a)\sim\mathcal{D}}\left[\frac{1}{2}\left(Q_\psi(s,a) - \left(r + \gamma\mathbb{E}_{s'\sim P}[V_{\bar{\psi}}(s')]\right)\right)^2\right]$$

where the soft value is defined as $V(s) = \mathbb{E}_{a\sim\pi}[Q(s,a) - \alpha\log\pi(a|s)]$.

The Actor seeks to minimize the expected KL divergence, which is equivalent to minimizing:

$$J_\pi(\theta) = \mathbb{E}_{s\sim\mathcal{D}}\left[\mathbb{E}_{a\sim\pi_\theta}[\alpha\log\pi_\theta(a|s) - Q_\psi(s,a)]\right]$$

Variables:

- α: Temperature parameter determining the relative importance of entropy (exploration) vs. reward maximization. Not to be confused with a learning rate.
- $\mathcal{D}$: Replay buffer containing transitions (s, a, r, s', d).
- $\bar{\psi}$: Target critic parameters, updated via Polyak averaging (also known as a "soft target update").

[1] In practice, since most optimizers perform gradient *descent*, this is typically implemented by minimizing $-L^{\text{CLIP}}$.

Automatic Entropy Tuning

Rather than treating α as a fixed hyperparameter, SAC can automatically adjust it by minimizing the following loss:

$$J(\alpha) = \mathbb{E}_{a\sim\pi_\theta}\left[-\alpha\left(\log\pi_\theta(a|s) + \bar{\mathcal{H}}\right)\right]$$

where $\bar{\mathcal{H}}$ is a target entropy (commonly set to $-\dim(\mathcal{A})$, the negative of the action space dimensionality). This is what you will implement in `compute_alpha_loss`.

SAC Update Pseudocode

1. Sample a minibatch of transitions (s, a, r, s', d) from $\mathcal{D}$.
2. **Critic Update**:
   - Sample next actions: $a' \sim \pi_\theta(\cdot|s')$.
   - Compute targets: $y = r + \gamma(1-d)\left(\min_{i=1,2} Q_{\bar{\psi}_i}(s', a') - \alpha\log\pi_\theta(a'|s')\right)$.
   - Update critics ψ_i by minimizing MSE: $(Q_{\psi_i}(s,a) - y)^2$.
3. **Actor Update**:
   - Use the reparameterization trick to sample actions: $a_\theta(s, \xi)$ where $\xi \sim \mathcal{N}(0, I)$.
   - Update θ by minimizing: $\alpha\log\pi_\theta(a_\theta|s) - \min_{i=1,2} Q_{\psi_i}(s, a_\theta)$.
4. **Temperature Update**: Update α by minimizing $J(\alpha)$ as described above.
5. **Target Update**: $\bar{\psi} \leftarrow \tau\psi + (1-\tau)\bar{\psi}$.

2 Test Your Implementations

To train the agents, use the following commands:

```
uv run python agents/ppo.py --env_id="Your_Chosen_Env"
```

```
uv run python agents/sac.py --env_id="Your_Chosen_Env"
```

This will print key diagnostics and returns to the terminal. If returns increase over time, you are on the right track. **Note**: Some default hyperparameters may have been chosen sub-optimally to encourage you to explore tuning in the next problem.

Once training is finished, you can use the Jupyter notebooks in `rrl_cw2/notebooks` to plot your results and render your learned policy.

To choose your environments, consult the Gymnasium documentation for a detailed overview of each environment, including action and state spaces as well as reward functions: https://gymnasium.farama.org/environments/mujoco/.

3 Hyperparameter Tuning

Hyperparameter tuning is a critical step in benchmarking. For this problem, you must tune **two** hyperparameters of your choosing for each algorithm. You must justify your choices in your final report.

Perform a simple grid search by choosing 3 reasonable values for each hyperparameter, resulting in a $3 \times 3 = 9$ runs per algorithm.

Tip: While this is a relatively small sweep, the runs can take significant time (especially on a modest laptop, but it is possible—we have tested this ourselves). We recommend chaining runs overnight using the `&&` operator:

```
uv run python agents/ppo.py --clip_coef=0.1 ...  && \
uv run python agents/ppo.py --clip_coef=0.2 ...
```

Tip 2: You may want to reduce the number of timesteps for SAC to keep overall runtime manageable. Somewhere in the range of 400k to 500k timesteps is reasonable. If you are still struggling with runtime, you can reduce further, but we advise against going below 200k timesteps.

4 Method Comparison

For the final evaluation, run both algorithms using your "best" hyperparameters across multiple seeds (**at least 3**) to compute confidence intervals. For example:

```
uv run python agents/ppo.py --clip_coef=best_value \
--num_minibatches=best_value --seed=1 --final-run && \
uv run python agents/ppo.py --clip_coef=best_value \
--num_minibatches=best_value --seed=2 --final-run ...
```

Use the `--final-run` flag to ensure outputs are saved to the correct directory for the plotting scripts.

Confidence intervals will be computed automatically in the plotting notebook. You are welcome to create your own plots if you wish to customise them further.

You should also render your learned policies so that you can evaluate them qualitatively. A notebook is provided for this purpose as well.

5 Write Your Report

Mark Distribution

The coursework is marked out of 100, broken down as follows:

- **Code implementation (24 marks)**: For each of the 8 functions you implement (4 for PPO, 4 for SAC), you will receive up to **3 marks**. Partial marks will be awarded if you were on the right track.
- **Report (76 marks)**: Detailed breakdown below.

Note that while we bias marks towards the report a successful implementation will significantly strengthen your report, as you will have working results to discuss.

Report Guidelines

You have freedom in how you present your findings, but your report should address the following key elements:

1. **Algorithm Introduction (10 marks)**: Introduce PPO and SAC, explaining the difference between off-policy and on-policy algorithms. In your own words, describe how these algorithms work. In particular, explain the update logic you have implemented in Problem 1. Try to build intuitions: why does the logic make sense?
2. **Relevance to Robotics (3 marks)**: Explain in your own words why PPO and SAC are suitable for robotics environments. Why have they become go-to baselines for applying RL in robotic settings?
3. **Environment Description (6 marks)**: Introduce the environments you chose to benchmark. Use the MDP formalism to describe observations, actions, and rewards. Why did you choose these particular environments? Did you select them based on any interesting properties?
4. **Hyperparameter Analysis (12 marks)**: Describe your hyperparameter sweeps and justify the hyperparameters you chose to tune. Highlight any interesting findings: how did tuning these hyperparameters affect performance, and why do you think they had the effect they did? (Based on your results from Problem 3).
5. **Results and Comparison (20 marks)**: Using your results, compare PPO and SAC across your environments. Comment on sample efficiency, training stability, wall-clock training time, and reliability across seeds. Also comment on the qualitative behaviour of the learned policies: what do they look like? Do you think they have adequately solved the task? Why or why not? (Based on your results from Problem 4)
6. **Proposed Robotics Task (25 marks)**: Propose a more complex robotics task that you personally find interesting. Introduce the problem setting and describe how you would set up an RL environment using the MDP or POMDP formalism. Which of the two algorithms you tested would you choose for this task, and why? Discuss potential

challenges you might encounter when applying these RL algorithms. Would you even choose to use RL, or would you opt for other methods such as those covered in optimal control or perhaps imitation learning? Justify your reasoning.

Important: Keep your report brief—**maximum 4 pages**.

Submitting Your Assignment

For your submission, upload the following to LEARN:

1. Your **report as a PDF**. If you worked as a pair then make that explicit on the PDF and put both or your student ID's at the top of the report. Each member of the pair still has to submit the report independently, it is fine if the submission are identical though.
2. Your **code repository as a `.tar.gz` file**. You can compress the repository directory by right-clicking in a file explorer and choosing the appropriate compression option, or via the terminal:

   ```
   tar -czvf archive_name.tar.gz directory_name/
   ```

   Make sure to include the rendered policy GIFs and any plots you made, these should be stored in the repository's notebook directory by default.

Important: Before you create the tarball, delete or move the `.venv` directory located at the root of the repository (i.e. `rrl_mujoco_cw2/.venv`). Note that this is a hidden directory, so to see it in the terminal use `ls -a`, or enable viewing hidden files in your file explorer. This directory is created by `uv` and holds installed dependencies—it can be approximately 5 GB in size.

References

Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, 2018. URL `https://arxiv.org/abs/1801.01290`.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL `https://arxiv.org/abs/1707.06347`.

Emanuel Todorov, Tom Erez, and Yuval Tassa. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems*, pages 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.